CAHIER DES CHARGES ET ARCHITECTURE LOGICIELLE

Application web de gestion des inscriptions des étudiants

ESGC Verechaguine AK

Version 1.0

Octobre 2026

Auteurs / équipe projet : à compléter

PARTIE 1 – CAHIER DES CHARGES

1. Contexte

L'ESGC Verechaguine AK enregistre aujourd'hui les inscriptions de ses étudiants à la main, avec des fiches papier et des registres, ce qui demande du temps au personnel et favorise les erreurs. L'administration souhaite donc disposer d'une application web pour enregistrer les étudiants, consulter leur liste, les rechercher et modifier leurs informations.

2. Objectifs

2.1 Objectif général

Concevoir et développer une application web qui permet à l'administration de l'école de numériser et de centraliser la gestion des inscriptions des étudiants.

2.2 Objectifs spécifiques

- Enregistrer un nouvel étudiant rapidement et sans erreur.
- Consulter la liste de tous les étudiants inscrits.
- Rechercher un étudiant par son nom, son prénom ou son numéro d'inscription.
- Modifier les informations d'un étudiant déjà inscrit.
- Garantir la sécurité et la fiabilité des informations.

3. Périmètre du projet

3.1 Inclus dans le projet

- **Connexion sécurisée** : les personnes autorisées de l'école se connectent avec un identifiant et un mot de passe. Les autres personnes ne peuvent pas accéder aux informations.
- **Gestion des étudiants** : le secrétariat peut inscrire un nouvel étudiant, afficher la liste de tous les étudiants, rechercher un étudiant et corriger ses informations en cas d'erreur ou de changement.
- **Base de données centralisée** : toutes les informations sont gardées au même endroit, au lieu d'être dispersées dans des fiches papier et des registres.
- **Utilisation sur tous les appareils** : l'application s'affiche correctement sur un ordinateur, une tablette ou un téléphone, depuis un navigateur internet.

3.2 Hors périmètre

- **Paiement en ligne des frais de scolarité** : l'application n'enregistre pas les paiements. Elle sert uniquement à gérer les inscriptions.
- **Espace personnel pour les étudiants** : les étudiants ne se connectent pas. Seul le personnel de l'école utilise l'application.
- **Notes, emplois du temps et présences** : ces sujets ne concernent pas l'inscription et ne sont pas traités.

- **Application mobile native** : il n'y a pas d'application à télécharger sur téléphone. On utilise le navigateur internet.

4. Description de l'existant

4.1 Fonctionnement actuel

Aujourd'hui, les inscriptions des étudiants se font entièrement à la main. Le secrétariat reçoit l'étudiant, remplit une fiche papier et la classe dans un dossier. Les informations sont parfois recopiées dans un registre ou dans un fichier. Il n'existe aucun outil informatique commun pour suivre les inscriptions.

4.2 Limites constatées

- **Perte de temps** : chaque inscription demande de remplir et de recopier plusieurs documents.
- **Erreurs** : les fautes d'écriture et les oublis sont fréquents lors de la saisie à la main.
- **Doublons** : un même étudiant peut être enregistré deux fois sans que personne ne s'en rende compte.
- **Recherche difficile** : retrouver un dossier oblige à feuilleter des registres ou des classeurs.
- **Modification compliquée** : corriger une information oblige à raturer ou à refaire une fiche.
- **Risque de perte** : un dossier papier peut être égaré, abîmé ou détruit.
- **Pas de vue d'ensemble** : il est difficile de savoir rapidement combien d'étudiants sont inscrits.

5. Parties prenantes

Partie prenante	Rôle
Administrateur	Gère les comptes des utilisateurs et dispose de tous les droits sur l'application.
Secrétariat	Enregistre, consulte, recherche et modifie les informations des étudiants.

6. Besoins fonctionnels

6.1 Authentification et gestion des accès

- Seules les personnes autorisées de l'école pourront utiliser l'application, avec un identifiant et un mot de passe.
- Chaque personne pourra se déconnecter quand elle a fini.

6.2 Enregistrement d'un étudiant

- Le secrétariat remplira une fiche avec les informations de l'étudiant (nom, prénom, date de naissance, téléphone, filière, niveau, etc.).
- L'application attribuera automatiquement un numéro d'inscription unique à chaque étudiant.
- Elle préviendra si une information obligatoire manque ou si elle est mal écrite, par exemple un numéro de téléphone incomplet.
- Elle préviendra aussi si l'étudiant semble déjà inscrit, pour éviter les doublons.
- À la fin, elle confirmera que l'inscription a bien été enregistrée.

6.3 Consultation de la liste des étudiants

- L'administration pourra afficher tous les étudiants inscrits dans une liste claire.
- La liste pourra être classée par nom ou par date d'inscription.
- En cliquant sur un étudiant, on verra toutes ses informations.

6.4 Recherche d'un étudiant

- On pourra chercher un étudiant en tapant son nom, son prénom ou son numéro d'inscription.
- On pourra aussi n'afficher que les étudiants d'une filière ou d'un niveau.
- Si personne ne correspond à la recherche, l'application l'indiquera clairement.

6.5 Modification des informations

- À partir de la fiche d'un étudiant, on pourra modifier ses informations, par exemple un nouveau numéro de téléphone ou une faute dans le nom.
- L'application demandera de confirmer avant d'enregistrer les changements.
- Elle gardera la date de la dernière modification et le nom de la personne qui l'a faite.

6.6 Fonctionnalités optionnelles (évolutions possibles)

- Supprimer ou archiver un étudiant.
- Télécharger la liste des étudiants (PDF ou Excel).
- Imprimer un reçu ou un certificat d'inscription.
- Voir des chiffres simples, comme le nombre d'inscrits par filière.

7. Besoins non fonctionnels

- **Facilité d'utilisation** : l'application doit être simple, avec des écrans clairs, pour que le personnel de l'école puisse s'en servir après une courte explication.
- **Utilisable partout** : elle doit fonctionner sur un ordinateur, une tablette ou un téléphone, avec les navigateurs courants (Chrome, Firefox, Edge).
- **Rapidité** : une page doit s’afficher en moins de 3 secondes en usage normal.
- **Sécurité** : seules les personnes autorisées peuvent entrer, les mots de passe sont protégés et l'application résiste aux attaques courantes.

- **Fiabilité** : les données sont sauvegardées régulièrement, pour ne rien perdre en cas de panne.
- **Confidentialité** : les informations personnelles des étudiants ne sont visibles que par le personnel autorisé.
- **Facilité d'évolution** : l'application est construite de manière organisée, pour pouvoir y ajouter facilement de nouvelles fonctions plus tard.

8. Planning prévisionnel

Étape	Ce qui est fait	Durée estimée
Analyse des besoins	Rencontrer l'administration et valider le cahier des charges	1 semaine
Conception	Préparer les écrans et l'organisation des informations	1 à 2 semaines
Développement	Construire l'application	3 à 4 semaines
Tests	Vérifier que tout fonctionne et corriger les erreurs	1 semaine
Mise en service	Installer l'application et former le personnel	1 semaine

La durée totale prévue est d’environ 7 à 9 semaines.

9. Contraintes

- **Contrainte de délai** : l'application doit être livrée dans le temps prévu au planning, idéalement avant le début des inscriptions de la prochaine année scolaire.
- **Contrainte de personnel** : le personnel de l'école n'est pas spécialiste en informatique, donc l'application doit rester très simple à utiliser.
- **Contrainte de matériel** : l'application doit fonctionner avec les ordinateurs, les tablettes et la connexion internet dont l'école dispose déjà.
- **Contrainte de sécurité** : les informations personnelles des étudiants doivent rester protégées et accessibles seulement au personnel autorisé.
- **Contrainte technique** : l'application sera une application web, utilisable depuis un navigateur sans installation sur chaque poste.

10. User stories

Authentification et gestion des accès

US1 : En tant que membre du secrétariat, je veux me connecter avec mon identifiant et mon mot de passe, afin que seules les personnes autorisées accèdent aux informations.

> ***Critère d'acceptation :*** Quand j'entre un identifiant et un mot de passe corrects, j'accède à l'application. Quand l'un des deux est faux, un message d'erreur s'affiche et l'accès est refusé.

US2 : En tant que membre du secrétariat, je veux me déconnecter quand j'ai fini, afin de protéger les informations quand je quitte mon poste.

> ***Critère d'acceptation :*** Après avoir cliqué sur « Se déconnecter », je reviens à la page de connexion et je ne peux plus voir les informations sans me reconnecter.

US3 : En tant qu'administrateur, je veux créer et gérer les comptes des personnes qui utilisent l'application, afin de décider qui y a accès.

> ***Critère d'acceptation :*** Je peux ajouter un nouveau compte avec un identifiant, un mot de passe et un rôle. La personne peut ensuite se connecter avec ce compte.

Enregistrement d'un étudiant

US4 : En tant que membre du secrétariat, je veux remplir une fiche d'inscription dans l'application, afin de ne plus écrire les informations à la main.

> ***Critère d'acceptation :*** Après avoir rempli la fiche et cliqué sur « Enregistrer », un message confirme l'inscription et l'étudiant apparaît dans la liste.

US5 : En tant que membre du secrétariat, je veux que l'application attribue automatiquement un numéro d'inscription, afin que chaque étudiant ait un numéro unique sans que j'aie à le chercher.

> ***Critère d'acceptation :*** Chaque nouvel étudiant reçoit un numéro d'inscription affiché sur sa fiche, et deux étudiants n'ont jamais le même numéro.

US6 : En tant que membre du secrétariat, je veux être prévenu si une information obligatoire manque ou est mal écrite, afin d'éviter les erreurs de saisie.

> ***Critère d'acceptation :*** Si je laisse un champ obligatoire vide ou si j'écris un numéro de téléphone incomplet, un message m'indique le champ à corriger et la fiche n'est pas enregistrée.

US7 : En tant que membre du secrétariat, je veux être prévenu si un étudiant semble déjà inscrit, afin d'éviter les doublons.

> ***Critère d'acceptation :*** Si j'entre une information déjà utilisée par un autre étudiant, comme le même e-mail, un message m'avertit avant l'enregistrement.

Consultation de la liste des étudiants

US8 : En tant que membre du secrétariat, je veux afficher la liste de tous les étudiants inscrits, afin de savoir en un coup d'œil qui est inscrit.

> ***Critère d'acceptation :*** La liste affiche pour chaque étudiant son numéro d'inscription, son nom, son prénom, sa filière et son niveau. Les étudiants récemment enregistrés y figurent.

US9 : En tant que membre du secrétariat, je veux classer la liste par nom ou par date d'inscription, afin de retrouver plus facilement un étudiant.

> ***Critère d'acceptation :*** Quand je choisis un classement, la liste se réorganise immédiatement dans l'ordre demandé.

US10 : En tant que membre du secrétariat, je veux ouvrir la fiche d'un étudiant, afin de voir toutes ses informations.

> ***Critère d'acceptation :*** Quand je clique sur un étudiant dans la liste, sa fiche s'ouvre avec toutes les informations enregistrées.

Recherche d'un étudiant

US11 : En tant que membre du secrétariat, je veux rechercher un étudiant par son nom, son prénom ou son numéro d'inscription, afin de retrouver son dossier rapidement.

> ***Critère d'acceptation :*** Quand je tape un nom, un prénom ou un numéro, seuls les étudiants correspondants s'affichent. Si personne ne correspond, un message clair l'indique.

US12 : En tant que membre du secrétariat, je veux n'afficher que les étudiants d'une filière ou d'un niveau, afin de consulter un groupe précis d'étudiants.

> ***Critère d'acceptation :*** Quand je choisis une filière ou un niveau, la liste ne montre que les étudiants concernés, et je peux revenir à la liste complète.

Modification des informations

US13 : En tant que membre du secrétariat, je veux modifier les informations d'un étudiant, afin de corriger une erreur ou de mettre à jour une information.

> ***Critère d'acceptation :*** Après avoir modifié une information sur la fiche et enregistré, la nouvelle information s'affiche immédiatement sur la fiche et dans la liste.

US14 : En tant que membre du secrétariat, je veux confirmer avant d'enregistrer une modification, afin d'éviter de changer une information par erreur.

> ***Critère d'acceptation :*** Avant l'enregistrement, l'application me demande de confirmer. Si je refuse, la fiche reste comme elle était.

PARTIE 2 – DOCUMENT D'ARCHITECTURE LOGICIELLE

Cette partie fait suite au cahier des charges et décrit l'architecture de l'application : architecture globale, modules, modèle d'information, aspects techniques, sécurité, déploiement, organisation du développement, risques et évolutions. Le contexte, les exigences et les acteurs sont déjà couverts par la partie 1.

1. Architecture globale du système

1.1 Vue d'ensemble

L'application est composée de trois blocs principaux :

- **L'interface (navigateur)** : les pages web que voit le personnel (connexion, formulaire d'inscription, liste, fiche, recherche). Elles sont écrites en HTML, CSS et JavaScript, avec Bootstrap pour l'adaptation aux ordinateurs, tablettes et téléphones.
- **Le serveur applicatif (Laravel, en PHP)** : il reçoit les demandes du navigateur, vérifie que la personne est autorisée, applique les règles (contrôle des champs, détection des doublons, attribution du numéro d'inscription), puis construit la page à afficher.
- **La base de données (MySQL)** : elle conserve les étudiants et les comptes des utilisateurs.

Navigateur (secrétariat, administrateur) → Serveur Laravel → Base de données MySQL

Il n'y a pas de service externe dans cette version. Le paiement, l'envoi de messages et le stockage de fichiers sont hors périmètre.

1.2 Principes d'architecture

- **Style architectural** : une application unique (monolithe) organisée en couches, selon le modèle MVC de Laravel : les modèles (les données), les vues (les pages) et les contrôleurs (la logique de chaque action). Ce choix est adapté à un petit projet avec peu d'utilisateurs, et il est simple à installer et à maintenir.
- **Communication** : le navigateur envoie des requêtes HTTP classiques (affichage d'une page, envoi d'un formulaire), et le serveur répond avec des pages complètes. Il n'y a pas d'API REST dans cette version.
- **Séparation des responsabilités** : l'affichage, la logique métier et l'accès aux données sont séparés, pour pouvoir modifier l'un sans casser les autres.
- **Sécurité** : toutes les pages, sauf la connexion, sont réservées aux utilisateurs connectés. Les droits dépendent du rôle (l'administrateur gère les comptes, le secrétariat gère les étudiants). Les données saisies sont toujours contrôlées côté serveur.

2. Conception des modules

2.1 Vue d’ensemble des modules

L'application est découpée en trois modules :

- **Gestion des comptes et des accès** : permet de se connecter et de se déconnecter, et permet à l'administrateur de gérer les comptes des utilisateurs.
- **Gestion des étudiants** : permet d'enregistrer un étudiant, d'afficher la liste, d'ouvrir une fiche et de modifier les informations.
- **Recherche et filtrage** : permet de retrouver un étudiant par nom, prénom ou numéro d'inscription, et de filtrer par filière ou par niveau.

Gestion des comptes et des accès → Gestion des étudiants → Recherche et filtrage

Le premier module contrôle l'accès aux deux autres : personne ne peut enregistrer ou consulter un étudiant sans être connecté. Le module de recherche s'appuie sur les données gérées par le module des étudiants.

2.2 Description détaillée des modules

2.2.1 Gestion des comptes et des accès

1. Rôle et responsabilité du module

Ce module garantit que seules les personnes autorisées utilisent l'application et que chacune n'a que les droits correspondant à son rôle. Il protège l'accès aux données personnelles des étudiants.

2. Fonctionnalités prises en charge

- Connexion avec un identifiant et un mot de passe.
- Déconnexion sécurisée.
- Création d'un compte par l'administrateur.
- Modification et désactivation d'un compte par l'administrateur.
- Contrôle des droits selon le rôle (administrateur ou secrétariat).

3. Entités du module

Utilisateur : une personne autorisée à se servir de l'application (administrateur ou membre du secrétariat).

Attribut	Type / Format	Description métier	Contraintes principales
id	Entier	Identifiant unique de l'utilisateur	Unique, obligatoire, automatique
nom	Texte	Nom complet de la personne	Obligatoire, 100 caractères maximum
identifiant	Texte	Nom utilisé pour se connecter	Unique, obligatoire
mot_de_passe	Texte (haché)	Mot de passe de connexion, jamais stocké en clair	Obligatoire, 8 caractères minimum
rôle	Texte	Rôle de la personne dans l'application	Valeurs autorisées : administrateur, secrétariat

Attribut	Type / Format	Description métier	Contraintes principales
actif	Oui / Non	Indique si le compte peut se connecter	Valeur par défaut : oui
créé_le	Date et heure	Date de création du compte	Renseigné automatiquement

4. Interfaces exposées par le module

- Page de connexion (lecture et envoi du formulaire), accessible à tous.
- Action de déconnexion, accessible aux utilisateurs connectés.
- Page de liste des comptes (lecture), réservée à l'administrateur.
- Formulaire de création et de modification d'un compte (écriture), réservé à l'administrateur.

5. Flux fonctionnels principaux

- *Connexion* : l'utilisateur saisit son identifiant et son mot de passe. Le serveur vérifie les informations et contrôle que le compte est actif. Si tout est correct, il ouvre la session et affiche la page d'accueil. Sinon, il affiche un message d'erreur.
- *Création d'un compte* : l'administrateur remplit le formulaire. Le serveur vérifie que l'identifiant n'existe pas déjà, protège le mot de passe et enregistre le compte. La personne peut ensuite se connecter.

6. Dépendances vers d’autres modules

Ce module ne dépend d'aucun autre. Les deux autres modules l'utilisent pour savoir qui est connecté et quels sont ses droits.

7. Règles métier et contraintes spécifiques

- L'identifiant d'un utilisateur est unique.
- Le mot de passe est protégé avant d'être enregistré, et personne ne peut le relire.
- Seul l'administrateur gère les comptes.
- Un compte désactivé ne peut plus se connecter.
- Toutes les pages, sauf la page de connexion, sont réservées aux utilisateurs connectés.
- **Sécurité (OWASP)** :
  - Blocage temporaire du compte après plusieurs échecs de connexion, contre les attaques par essais répétés.
  - Mots de passe hachés avec un algorithme moderne et session protégée (renouvelée à la connexion, expirant après inactivité).
  - Messages d'erreur qui ne révèlent pas si c'est l'identifiant ou le mot de passe qui est faux.
  - Jeton de protection contre les requêtes falsifiées (CSRF) dans chaque formulaire, et contrôle des droits côté serveur à chaque demande.
- **Accessibilité (WCAG 2.2 AA)** :
  - Formulaires de connexion et de gestion des comptes avec des étiquettes visibles, utilisables entièrement au clavier.

- Messages d'erreur écrits en toutes lettres, pas seulement signalés par une couleur.
- Le gestionnaire de mots de passe du navigateur peut remplir le formulaire, et le mot de passe n'a pas à être retapé inutilement.

2.2.2 Gestion des étudiants

1. Rôle et responsabilité du module

Ce module est le cœur de l'application. Il garantit que chaque étudiant est enregistré une seule fois, avec des informations complètes, correctes et à jour. Il protège les données personnelles des étudiants.

2. Fonctionnalités prises en charge

- Enregistrement d'un nouvel étudiant avec un numéro d'inscription généré automatiquement.
- Contrôle des informations saisies (champs obligatoires, formats du téléphone, de l'e-mail et de la date).
- Détection des doublons avant l'enregistrement.
- Affichage de la liste des étudiants, avec pagination et tri par nom ou par date d'inscription.
- Affichage de la fiche détaillée d'un étudiant.
- Modification des informations d'un étudiant, avec confirmation avant l'enregistrement.
- Conservation de la date et de l'auteur de la dernière modification.

3. Entités du module

Étudiant : une personne inscrite à l'école, avec les informations de sa fiche d'inscription.

Attribut	Type / Format	Description métier	Contraintes principales
id	Entier	Identifiant technique de l'étudiant	Unique, obligatoire, automatique
numéro_inscription	Texte (ex. 2026-0001)	Numéro d'inscription visible par les utilisateurs	Unique, obligatoire, généré automatiquement, non modifiable
nom	Texte	Nom de famille	Obligatoire, 100 caractères maximum
prénom	Texte	Prénom	Obligatoire, 100 caractères maximum
sexe	Texte	Sexe de l'étudiant	Obligatoire, valeurs autorisées : masculin, féminin
date_naissance	Date	Date de naissance	Obligatoire, ne peut pas être dans le futur

Attribut	Type / Format	Description métier	Contraintes principales
lieu_naissance	Texte	Ville ou lieu de naissance	Facultatif, 100 caractères maximum
adresse	Texte	Adresse de résidence	Facultatif, 255 caractères maximum
téléphone	Texte	Numéro de téléphone de l'étudiant	Obligatoire, format valide
email	Texte	Adresse e-mail de l'étudiant	Facultatif, format valide, unique s'il est renseigné
filière	Texte	Filière ou classe d'inscription	Obligatoire
niveau	Texte	Niveau d'études (ex. Licence 1)	Obligatoire
année_académique	Texte (ex. 2026-2027)	Année pour laquelle l'étudiant est inscrit	Obligatoire, format valide
date_inscription	Date	Date de l'inscription	Obligatoire, renseignée par défaut avec la date du jour
tuteur_nom	Texte	Nom du tuteur ou de la personne à contacter	Facultatif, 100 caractères maximum
tuteur_téléphone	Texte	Téléphone du tuteur	Facultatif, format valide
créé_le	Date et heure	Date de création de la fiche	Renseigné automatiquement
modifié_le	Date et heure	Date de la dernière modification	Mis à jour automatiquement
modifié_par	Référence vers un utilisateur	Personne ayant fait la dernière modification	Renseigné automatiquement

4. Interfaces exposées par le module

- Formulaire d'inscription d'un étudiant (écriture), pour le secrétariat et l'administrateur.
- Liste des étudiants avec tri et pagination (lecture).
- Fiche détaillée d'un étudiant (lecture).
- Formulaire de modification d'un étudiant (lecture et écriture), avec une page de confirmation avant l'enregistrement.

Il n'y a pas de suppression dans cette version, car elle fait partie des fonctionnalités optionnelles.

5. Flux fonctionnels principaux

- *Enregistrement d'un étudiant* : le secrétariat remplit le formulaire d'inscription. Le serveur contrôle chaque champ, vérifie qu'aucun étudiant n'a déjà le même e-mail, puis génère le numéro d'inscription et enregistre la fiche. Il affiche enfin un message de confirmation. En cas d'erreur, le formulaire se réaffiche avec les champs à corriger, sans perdre ce qui a été saisi.
- *Modification d'un étudiant* : le secrétariat ouvre la fiche, puis le formulaire de modification. Il change une information, et le serveur applique les mêmes contrôles qu'à l'inscription. Après confirmation, il enregistre la modification et note la date et l'auteur du changement.

6. Dépendances vers d'autres modules

- **Gestion des comptes et des accès** : pour vérifier que la personne est connectée, contrôler son rôle et connaître l'auteur d'une modification.

Le module de recherche et de filtrage s'appuie à son tour sur les données de ce module.

7. Règles métier et contraintes spécifiques

- Le numéro d'inscription est unique, généré par l'application et ne peut pas être modifié.
- Un e-mail ne peut être associé qu'à un seul étudiant.
- Les champs obligatoires doivent être remplis, et les formats (téléphone, e-mail, dates) sont vérifiés.
- Toute modification est confirmée avant d'être enregistrée.
- La date et l'auteur de la dernière modification sont toujours conservés.
- Les données personnelles ne sont visibles que par le personnel autorisé.
- **Sécurité (OWASP)** :
  - Les données saisies sont toujours contrôlées côté serveur, jamais seulement dans le navigateur.
  - Les requêtes vers la base de données n'utilisent que des paramètres sécurisés, ce qui protège contre l'injection SQL.
  - Les informations affichées sont systématiquement échappées, ce qui protège contre les attaques par injection de script (XSS).
  - Chaque formulaire contient un jeton de protection contre les requêtes falsifiées (CSRF).
  - Seuls les champs autorisés sont enregistrés lors d'une modification, pour empêcher la modification de champs interdits (comme le numéro d'inscription).
  - L'accès aux fiches est vérifié à chaque demande selon le rôle de l'utilisateur, et pas seulement en cachant des boutons.
- **Accessibilité (WCAG 2.2 AA)** :
  - Chaque champ du formulaire a une étiquette visible, et les champs obligatoires sont indiqués clairement.
  - Les erreurs de saisie sont décrites en toutes lettres à côté du champ concerné et résumées en haut du formulaire.
  - Tout se fait au clavier, avec un contour de focus visible, y compris le tri et la pagination de la liste.

- Le contraste du texte est d'au moins 4,5 contre 1, et l'information ne repose jamais sur la couleur seule.
- Le tableau de la liste est correctement structuré (titres de colonnes) pour les lecteurs d'écran.
- Les informations déjà saisies ne sont pas à ressaisir lors de la confirmation d'une modification.

2.2.3 Recherche et filtrage

1. Rôle et responsabilité du module

Ce module permet de retrouver rapidement un étudiant sans parcourir toute la liste. Il garantit que les résultats affichés sont exacts, que la recherche reste simple pour le secrétariat et qu'elle ne peut pas être détournée pour accéder à des données non autorisées.

2. Fonctionnalités prises en charge

- Recherche d'un étudiant par nom, prénom ou numéro d'inscription.
- Recherche partielle : taper le début d'un nom suffit.
- Recherche qui ignore les majuscules et les accents (« Élodie » est trouvée avec « elodie »).
- Filtrage de la liste par filière ou par niveau.
- Combinaison de la recherche et des filtres.
- Pagination des résultats.
- Message clair quand aucun étudiant ne correspond.
- Retour à la liste complète en un clic.

3. Entités du module

Ce module ne possède pas de données propres : il ne stocke rien et travaille sur l'entité **Étudiant**, définie dans le module 2.2.2. Il utilise seulement des critères temporaires, saisis par l'utilisateur et oubliés après l'affichage des résultats.

Critères de recherche (non enregistrés)

Attribut	Type / Format	Description métier	Contraintes principales
terme	Texte	Mot saisi dans la barre de recherche (nom, prénom ou numéro)	Facultatif, 100 caractères maximum
filière	Texte	Filière choisie dans le filtre	Facultatif, valeur existante uniquement
niveau	Texte	Niveau choisi dans le filtre	Facultatif, valeur existante uniquement
page	Entier	Page de résultats demandée	Facultatif, 1 par défaut, valeur positive

4. Interfaces exposées par le module

- Barre de recherche, placée en haut de la liste des étudiants (lecture).

- Filtres par filière et par niveau (lecture).
- Liste des résultats, avec pagination (lecture).
- Message « Aucun étudiant trouvé » avec un lien pour revenir à la liste complète.

Le module ne permet aucune écriture : on y cherche, on n'y modifie rien.

5. Flux fonctionnels principaux

- *Recherche par nom* : le secrétariat tape un nom dans la barre de recherche et valide. Le serveur contrôle la saisie, cherche les étudiants dont le nom, le prénom ou le numéro correspond, et affiche la liste des résultats avec le nombre trouvé. Le secrétariat clique sur un étudiant pour ouvrir sa fiche.
- *Filtrage* : le secrétariat choisit une filière, éventuellement un niveau, et valide. Le serveur affiche uniquement les étudiants concernés. Les filtres choisis restent visibles pour que l'utilisateur sache ce qui est appliqué.
- *Aucun résultat* : le serveur affiche un message clair et propose de revenir à la liste complète.

6. Dépendances vers d'autres modules

- **Gestion des comptes et des accès** : pour vérifier que la personne est connectée avant toute recherche.
- **Gestion des étudiants** : pour lire les fiches des étudiants et ouvrir la fiche d'un résultat.

7. Règles métier et contraintes spécifiques

- La recherche ne renvoie que des étudiants existants et seulement à des utilisateurs connectés.
- Les résultats sont paginés pour garder un affichage rapide, en moins de 3 secondes.
- Les filtres n'acceptent que des valeurs existantes.
- **Sécurité (OWASP)** :
  - Le texte saisi est contrôlé côté serveur (longueur limitée) et envoyé à la base uniquement par des requêtes paramétrées, contre l'injection SQL.
  - Le texte recherché est échappé quand il est réaffiché dans la page, contre les attaques XSS.
  - Les valeurs des filtres et le numéro de page sont validés, pour refuser toute valeur inattendue.
  - L'accès à la recherche est vérifié à chaque demande, comme pour les autres pages.
- **Accessibilité (WCAG 2.2 AA)** :
  - Le champ de recherche et chaque filtre ont une étiquette visible et claire, pas seulement un texte grisé dans le champ.
  - Tout peut se faire au clavier (recherche, filtres, ouverture d'une fiche, pagination), avec un contour de focus bien visible.
  - Le nombre de résultats et le message « Aucun étudiant trouvé » sont annoncés aux lecteurs d'écran.
  - Le contraste entre le texte et le fond est d'au moins 4,5 contre 1, et l'information ne repose jamais sur la couleur seule.
  - Les boutons et liens ont une taille suffisante (au moins 24 par 24 pixels), pour être utilisables sur tablette et téléphone.

– La page reste utilisable quand on zoome à 200 %, sans perdre d'information.

3. Vue globale du modèle d'information

3.1 Inventaire des entités par module

Module	Entités principales gérées
Gestion des comptes et des accès	Utilisateur
Gestion des étudiants	Étudiant
Recherche et filtrage	Aucune entité propre (travaille sur Étudiant, avec des critères temporaires non enregistrés)

L'application gère donc deux entités enregistrées en base : Utilisateur et Étudiant. Leurs attributs sont décrits dans les sections 2.2.1 et 2.2.2.

3.2 Relations entre entités

Utilisateur (1) ——— modifie ———> (0..n) Étudiant

- Un utilisateur peut modifier plusieurs fiches d'étudiants, ou aucune.
- Une fiche d'étudiant garde la trace d'un seul utilisateur : le dernier qui l'a modifiée (attribut modifié_par).
- Un étudiant n'a pas de lien avec un compte. Les étudiants ne se connectent pas à l'application.

Le modèle est volontairement très simple : deux tables et une seule relation. Cela correspond au périmètre de la première version. Si on ajoutait plus tard les paiements ou les notes, il faudrait de nouvelles entités liées à Étudiant.

4. Architecture applicative et technique

4.1 Couches logicielles

L'application suit l'organisation de Laravel, en cinq couches :

- **Présentation** : les pages affichées dans le navigateur (vues Blade, HTML, CSS, Bootstrap). Cette couche montre les informations et reçoit les saisies, sans appliquer de règle métier.
- **Routes et contrôleurs** : ils reçoivent chaque demande du navigateur (par exemple « afficher la liste » ou « enregistrer un étudiant »), vérifient les droits de l'utilisateur et déclenchent la bonne action.
- **Validation et logique métier** : elle contrôle les champs saisis, détecte les doublons, génère le numéro d'inscription et applique les règles propres à l'école.
- **Accès aux données** : les modèles (Utilisateur, Étudiant) lisent et écrivent dans la base de données, toujours par des requêtes paramétrées.

- **Infrastructure** : le serveur web, PHP et la base de données MySQL, sur lesquels tourne l'application.

Une couche ne communique qu'avec celle qui est juste en dessous. On peut ainsi modifier l'affichage sans toucher aux règles métier.

4.2 Technologies et frameworks

Domaine	Technologie	Pourquoi ce choix
Interface utilisateur	HTML, CSS, JavaScript, vues Blade et Bootstrap	Pages simples construites par le serveur, adaptées à tous les écrans, avec des composants accessibles
Backend	PHP et Laravel (dernière version stable)	Fournit la connexion, les rôles, la validation et les protections de sécurité déjà prêtes
Base de données	MySQL	Données très structurées, bien intégrée à Laravel et proposée par la plupart des hébergeurs
Serveur web	Apache	Serveur très répandu, bien documenté et compatible avec Laravel
Communication	HTTP avec HTTPS obligatoire	Échanges classiques entre le navigateur et le serveur, chiffrés pour protéger les données
Authentification	Système intégré de Laravel	Gère les sessions et le chiffrement des mots de passe sans code à écrire soi-même
Gestion du code	Composer pour les dépendances PHP et Git pour le suivi	Outils standards de l'écosystème Laravel
Services externes	Aucun	Hors périmètre dans cette version

5. Sécurité et qualités techniques

5.1 Sécurité

La sécurité suit les recommandations de l'OWASP, en particulier sa liste des dix risques les plus courants pour les applications web. Le tableau montre les mesures prévues pour chaque risque.

Risque OWASP	Mesure prévue
Contrôle d’accès défaillant	Chaque demande est vérifiée côté serveur selon le rôle de l'utilisateur. Un secrétariat ne peut pas accéder à la gestion des comptes, même en tapant l'adresse de la page.

Risque OWASP	Mesure prévue
Défaillances cryptographiques	HTTPS obligatoire sur toutes les pages, mots de passe hachés avec un algorithme moderne (bcrypt ou Argon2), cookies de session sécurisés.
Injection (SQL, scripts)	Requêtes à la base toujours paramétrées. Toute information affichée est échappée automatiquement par Blade, contre les attaques XSS.
Conception non sécurisée	Droits limités au strict nécessaire, contrôles de saisie côté serveur, pas de champ modifiable sans autorisation (ex. numéro d'inscription).
Mauvaise configuration de sécurité	Mode débogage désactivé en production, messages d'erreur sans détail technique, fichiers de configuration et mots de passe de la base placés hors de l'accès public.
Composants vulnérables ou obsolètes	Laravel, PHP et les bibliothèques sont maintenus à jour, et les dépendances sont vérifiées régulièrement avec Composer.
Défaillances d'authentification	Blocage temporaire après plusieurs échecs de connexion, session renouvelée à chaque connexion, expiration après inactivité, message d'erreur qui ne révèle pas si c'est l'identifiant ou le mot de passe qui est faux.
Défaillances d'intégrité	Jeton de protection CSRF dans chaque formulaire, dépendances installées uniquement depuis des sources officielles.
Défaillances de journalisation et de surveillance	Les actions sensibles sont enregistrées (voir ci-dessous).
Falsification de requêtes côté serveur (SSRF)	Peu concerné dans cette version : l'application n'appelle aucun service externe.

- **Authentification et rôles** : la connexion se fait par identifiant et mot de passe. Il y a deux rôles : l'administrateur gère les comptes, le secrétariat gère les étudiants. Les droits sont contrôlés à chaque action.

Protection des données sensibles :

- *En transit* : toutes les communications passent par HTTPS.
- *Au repos* : les mots de passe sont hachés, et la base de données n'est pas accessible depuis internet. Seul le serveur de l'application peut s'y connecter.
- Les données personnelles des étudiants ne sont visibles que par les utilisateurs connectés et autorisés.
- **Journalisation (audit)** : l'application enregistre la date, l'utilisateur et l'action pour les événements sensibles : connexions réussies et échouées, création ou modification d'un compte, enregistrement et modification d'un étudiant. Les journaux ne contiennent jamais de mot de passe.

5.2 Autres qualités techniques

Accessibilité (WCAG 2.2, niveau AA) :

- Toutes les fonctions sont utilisables au clavier, avec un contour de focus visible.
- Chaque champ de formulaire a une étiquette visible. Les erreurs sont décrites par du texte clair, pas seulement par une couleur.
- Le contraste entre le texte et le fond est d'au moins 4,5 contre 1.
- Les boutons et liens ont une taille suffisante pour le tactile (au moins 24 par 24 pixels).
- La page reste utilisable avec un zoom à 200 %, sur téléphone comme sur ordinateur.
- La structure des pages (titres, tableaux, formulaires) est correcte pour que les lecteurs d'écran s'y repèrent.
- L'utilisateur n'a jamais à ressaisir une information qu'il a déjà donnée lors d'une même étape.

Performance :

- Liste et résultats de recherche paginés.
- Index sur les colonnes les plus utilisées (nom, prénom, numéro d'inscription).
- Chargement des seules données nécessaires à chaque page.
- Objectif : page affichée en moins de 3 secondes en usage normal.

Robustesse :

- Gestion des erreurs : l'utilisateur voit un message compréhensible, et le détail technique va dans les journaux.
- Les formulaires se réaffichent avec les informations déjà saisies en cas d'erreur.
- Les enregistrements en base sont faits par transactions, pour éviter une fiche à moitié enregistrée.
- Sauvegarde régulière de la base de données, avec test de restauration.

Observabilité de base :

- Journaux applicatifs de Laravel pour les erreurs et les événements importants.
- Consultation des journaux par l'administrateur technique, sans accès pour le secrétariat.

6. Déploiement et environnements

6.1 Environnements prévus

Environnement	Usage principal	Où il tourne
Développement	Écrire et tester le code au quotidien	Poste de chaque développeur, avec des données fictives
Test (préproduction)	Vérifier que l'application fonctionne avant la mise en service, et la présenter à l'école pour validation	Serveur cloud (VPS), configuré comme la production
Production	Utilisation réelle par l'administration	Serveur cloud (VPS)

Les données réelles des étudiants ne sont utilisées qu'en production. Les environnements de développement et de test n'ont que des données fictives, pour protéger les informations personnelles. Si le projet reste académique, l'environnement de test peut servir de démonstration et la production n'être mise en place qu'au moment de la mise en service.

6.2 Vue de déploiement

L'application est déployée sur un serveur cloud (VPS) unique, qui regroupe le serveur web, l'application et la base de données.

Navigateur du personnel → Internet (HTTPS) → Pare-feu → Serveur VPS [Apache + PHP + Laravel → MySQL]

- **Serveur applicatif** : Apache reçoit les demandes du navigateur et les transmet à PHP, qui exécute Laravel. Seul le dossier public de l'application est exposé. Le code et les fichiers de configuration restent hors de l'accès web.
- **Base de données** : MySQL est installée sur le même serveur. Elle n'accepte que les connexions locales et n'est jamais accessible depuis internet. L'application s'y connecte avec un compte dédié, aux droits limités.
- **Réseau et sécurité** : un pare-feu ne laisse ouverts que les ports nécessaires (HTTPS pour les utilisateurs, accès d'administration sécurisé pour l'équipe technique). L'accès sans chiffrement (HTTP) est redirigé vers HTTPS. Un certificat HTTPS gratuit et renouvelé automatiquement (Let's Encrypt) protège les échanges.
- **Nom de domaine** : une adresse web dédiée, par exemple une adresse propre à l'école, qui pointe vers le serveur.
- **Sauvegardes** : une sauvegarde automatique de la base de données est faite chaque jour et conservée sur un espace distinct du serveur. La restauration est testée régulièrement.
- **Services externes critiques** : aucun dans cette version.

Mise à jour de l'application. Les nouvelles versions sont d'abord déployées en test, vérifiées, puis installées en production, de préférence en dehors des heures de travail du secrétariat. Le système, PHP, Apache, MySQL et les bibliothèques sont mis à jour régulièrement pour corriger les failles de sécurité.

7. Organisation du développement et gestion du code

7.1 Gestion du code source

- **Outil de suivi** : Git, qui conserve l'historique de toutes les modifications et permet de revenir en arrière si besoin.
- **Plateforme d'hébergement du code** : GitHub, avec un dépôt privé pour que le code ne soit pas public.
- **Structure du dépôt** : le dépôt suit l'organisation standard de Laravel. Il n'y a pas de séparation entre frontend et backend, puisque les pages sont construites par le serveur.

Dossier	Contenu
app/	Code de l’application : contrôleurs, modèles et règles métier
resources/views/	Pages affichées (vues Blade)

Dossier	Contenu
routes/	Liste des adresses de l'application et des actions associées
database/	Migrations (création des tables) et données fictives de test
public/	Seul dossier accessible depuis internet (styles, scripts, images)
tests/	Tests automatiques
docs/	Documentation : cahier des charges, architecture, guide d'installation

Fichiers à ne jamais enregistrer dans Git : le fichier de configuration .env, qui contient les mots de passe et les clés secrètes, et les dossiers de dépendances. Ils sont listés dans le fichier .gitignore. Un fichier d'exemple .env.example, sans aucun secret, explique la configuration à prévoir.

7.2 Stratégie de branches et contributions

Branches principales

- *main* : le code stable, celui qui est installé en production. Personne n'y écrit directement.
- *develop* : le code en cours d'assemblage, qui sert à l'environnement de test.

Branches de fonctionnalité (feature branches)

- Chaque nouvelle fonction a sa propre branche, créée à partir de develop, avec un nom clair, par exemple feature/recherche-etudiant ou feature/connexion.
- Une branche correspond à une user story (ou à un petit groupe de stories), ce qui relie le code aux cartes Trello.
- Une fois terminée, la branche est fusionnée dans develop, puis supprimée.

Processus de contribution

- Le développeur crée une branche à partir de develop et y travaille.
- Il enregistre ses modifications par petites étapes (commits), avec des messages clairs, par exemple « Ajout de la recherche par nom ».
- Il ouvre une pull request vers develop, c'est-à-dire une demande de fusion.
- Un autre membre de l'équipe relit le code (revue de code). Il vérifie les règles métier, la sécurité (OWASP) et l'accessibilité (WCAG 2.2 AA).
- Après validation et réussite des tests automatiques, la branche est fusionnée dans develop.
- Quand une version est prête et validée en test, develop est fusionné dans main, puis installé en production.

Dans le cas d'un projet réalisé seul, la revue de code est remplacée par une relecture personnelle avant la fusion, mais le reste du processus reste identique.

8. Risques, limites et évolutions

8.1 Risques principaux

Risque	Conséquence possible	Mesure pour le réduire
Panne du serveur unique (VPS)	L'application n'est plus accessible, et le secrétariat revient au papier le temps de la réparation	Sauvegardes quotidiennes, surveillance du serveur, procédure de remise en route documentée
Perte de données	Fiches d'étudiants perdues ou abîmées	Sauvegarde automatique sur un espace distinct, test régulier de restauration
Attaque ou fuite de données personnelles	Données des étudiants lues ou modifiées par une personne non autorisée	Application des recommandations OWASP, HTTPS, mots de passe hachés, journalisation, mises à jour régulières
Failles dans les composants (PHP, Laravel, bibliothèques)	Exploitation d'une faille connue	Mises à jour régulières et vérification des dépendances
Mauvaise prise en main par le personnel	Erreurs de saisie, refus d'utiliser l'application	Interface simple, formation courte, guide d'utilisation
Résistance au changement	Maintien des fiches papier en parallèle, donc deux sources d'information	Accompagnement, période de transition, puis arrêt du papier
Délais ou ressources limités	Retard sur le planning de 7 à 9 semaines	Priorité aux fonctions demandées par l'énoncé (user stories principales), options reportées
Coupure de connexion internet à l'école	Application inaccessible, car elle est hébergée en ligne	Connexion de secours (par exemple un téléphone en partage de connexion)
Compétences techniques limitées de l'équipe	Qualité ou sécurité insuffisante	Utilisation des outils intégrés à Laravel, revues de code

8.2 Limites connues de la version actuelle

- **Pas de suppression ni d'archivage** : un étudiant ne peut pas être retiré de la liste.
- **Pas d'export ni d'impression** : la liste ne peut pas être téléchargée en PDF ou Excel, et il n'y a pas de reçu d'inscription.
- **Pas de statistiques** : aucun tableau de bord, par exemple le nombre d'inscrits par filière.

- **Pas de pièces jointes** : on ne peut pas ajouter de photo, de pièce d'identité ou de diplôme à la fiche.
- **Filière et niveau en simple texte** : ils ne sont pas gérés dans des listes officielles, ce qui peut créer des écritures différentes pour une même filière.
- **Un seul serveur** : sans reprise automatique en cas de panne.
- **Pas d'accès pour les étudiants** : seul le personnel de l'école utilise l'application.
- **Pas d'historique détaillé des modifications** : seules la date et l'auteur de la dernière modification sont conservés.
- **Pas de journal visible dans l'application** : les journaux de sécurité sont accessibles par l'équipe technique, pas par l'administrateur depuis l'application.
- **Pas de paiement, de notes, d'emplois du temps ni de présences** : ils sont hors périmètre.

8.3 Pistes d'évolution

Nouvelles fonctionnalités

- Suppression ou archivage d'un étudiant.
- Export de la liste en PDF ou Excel et impression d'un reçu ou d'un certificat d'inscription.
- Tableau de bord avec des chiffres simples par filière et par niveau.
- Ajout de pièces jointes (photo, documents).
- Paiement en ligne des frais de scolarité.
- Espace personnel pour les étudiants.
- Gestion des notes, des emplois du temps et des présences.
- Envoi automatique de notifications par e-mail ou SMS.

Améliorations des modules existants

- Listes de filières et de niveaux gérées par l'administrateur, pour éviter les écritures différentes.
- Historique complet des modifications de chaque fiche.
- Consultation des journaux de sécurité depuis l'application.
- Recherche plus avancée, par exemple par année académique ou par date d'inscription.

Améliorations techniques

- Deuxième serveur ou sauvegarde en continu pour reprendre automatiquement en cas de panne.
- Authentification renforcée, avec un code supplémentaire envoyé à la connexion.
- Surveillance automatique du serveur avec alertes.
- Interface plus dynamique (React ou Vue) si le nombre d'utilisateurs augmente fortement.
- Passage à une API pour connecter d'autres outils de l'école.